

Unit Test Generation using Generative AI : A Comparative Performance Analysis of Autogeneration Tools

Shreya Bhatia*
shreya20542@iiitd.ac.in
IIIT Delhi
Delhi, India

Dhruv Kumar
dhruv.kumar@iiitd.ac.in
IIIT Delhi
Delhi, India

Tarushi Gandhi*
tarushi20579@iiitd.ac.in
IIIT Delhi
Delhi, India

Pankaj Jalote
jalote@iiitd.ac.in
IIIT Delhi
Delhi, India

Abstract

Generating unit tests is a crucial task in software development, demanding substantial time and effort from programmers. The advent of Large Language Models (LLMs) introduces a novel avenue for unit test script generation. This research aims to experimentally investigate the effectiveness of LLMs, specifically exemplified by ChatGPT, for generating unit test scripts for Python programs, and how the generated test cases compare with those generated by an existing unit test generator (Pynguin). For experiments, we consider three types of code units: 1) Procedural scripts, 2) Function-based modular code, and 3) Class-based code. The generated test cases are evaluated based on criteria such as coverage, correctness, and readability. Our results show that ChatGPT's performance is comparable with Pynguin in terms of coverage, though for some cases its performance is superior to Pynguin. We also find that about a third of assertions generated by ChatGPT for some categories were incorrect. Our results also show that there is minimal overlap in missed statements between ChatGPT and Pynguin, thus, suggesting that a combination of both tools may enhance unit test generation performance. Finally, in our experiments, prompt engineering improved ChatGPT's performance, achieving a much higher coverage.*These authors contributed equally.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → **Artificial intelligence**.

Keywords

Large Language Models, Unit Test Generation, ChatGPT, Generative AI

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

LLM4Code '24, April 20, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0579-3/24/04
<https://doi.org/10.1145/3643795.3648396>

ACM Reference Format:

Shreya Bhatia, Tarushi Gandhi, Dhruv Kumar, and Pankaj Jalote. 2024. Unit Test Generation using Generative AI : A Comparative Performance Analysis of Autogeneration Tools. In *2024 International Workshop on Large Language Models for Code (LLM4Code '24)*, April 20, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3643795.3648396>

1 Introduction

Unit testing is an integral part of software development as it helps catch errors early in the development process. Creating and maintaining effective unit tests manually is a notably laborious and time-consuming task. To address the difficulties inherent in manual test creation, various methodologies for automating the unit test generation process have been proposed by researchers. Common approaches in this field include search-based [4, 5, 9, 13, 31], constraint-based [23, 28], or random-based [8, 27] techniques, all aiming to generate a suite of unit tests with the primary goal of enhancing coverage in the targeted software. However, when compared to tests that are manually created, automated tests produced by these techniques could be less readable and comprehensible [14, 16]. This shortcoming makes it difficult for testers with little experience to learn and hinders the adoption of these strategies, especially for beginners. As a result, developers might be hesitant to include these automated tests straight into their workflows. To address these concerns, recent efforts have explored the use of advanced deep learning (DL) techniques, particularly large language models (LLMs), for unit test generation [11, 18, 19].

Advancements in Large Language Models (LLMs), exemplified by OpenAI's ChatGPT (GPT-3.5), showcase enhanced capabilities in routine tasks like question answering, translation, and text/code generation, rivaling human-like understanding. Unlike other LLMs such as BART[20] and BERT[10], ChatGPT [6] incorporates reinforcement learning from human feedback (RLHF) [7] and a larger model scale, improving generalization and alignment with human intention. Widely utilized in daily activities, ChatGPT is crucial for tasks like text generation, language translation, and automated customer support. Beyond daily use, large language models are increasingly applied in software engineering tasks, including code generation and summarization[3, 32]. These models can also facilitate the generation of unit test cases, streamlining software validation processes[19, 29, 34].

This paper seeks to explore the advantages and drawbacks of test suites produced by Large Language Models. We focus particularly on ChatGPT as a representative of the LLMs. Furthermore, we want to explore the potential synergy of integrating existing unit test generators such as Pynguin [21] with Large Language Model (LLM)-based approaches to enhance overall performance.

We evaluate the quality of unit tests generated by ChatGPT compared to Pynguin. Based on the code structure, we classify a sample Python code into 3 categories: 1) Procedural scripts, where code does not have classes or functions. 2) Function-based modular code is where there are clear definitions of functions which are standalone and act like independent units of code. and 3) Class-based modular code, which is structured around classes and objects, as the primary units of organisation.

We curated a dataset comprising 60 Python projects, categorizing them into 20 projects per category, each with a complete executable environment. In our study, we focus on a designated core module from each project, ranging between 100-300 lines of code, selected based on factors such as cyclomatic complexity, function count, and file interdependency. We then generate unit tests for the selected modules by prompting them as input to ChatGPT and compare them with unit tests generated by Pynguin. We aim to address the following research questions:

- **RQ1 (Comparative Performance):** How does ChatGPT compare with Pynguin in generating unit tests?
- **RQ2 (Performance Saturation and Iterative Improvement):** How does the effectiveness of test cases generated by ChatGPT improve/change over multiple iterations of prompting?
- **RQ3 (Quality Assessment):** How correct are the assertions generated by ChatGPT, and what percentage of assertions align with the intended functionality of the code?
- **RQ4 (Combining Tools for Improved Performance):** Can a combination of ChatGPT and Pynguin enhance the overall performance of unit test generation, in terms of coverage and effectiveness?

This paper validates the findings from existing work [19] which is very important in the rapidly evolving landscape of LLMs and additionally explores research questions not covered in the existing work.

The rest of the paper is organized as follows: We explain the methodology in §2 followed by results in §3. We discuss related work in §4 and conclude in §5.

2 Methodology

In this section, we discuss the systematic approach undertaken to compare the performance of various unit test generation tools, including the selection of code samples, their categorisation, tool choices, and evaluation metrics.

2.1 Categorisation Based on Code Structure

The three delineated categories served to capture varying levels of code organisation:

Category 1 (Procedural Scripts): Code samples are characterised by procedural scripts lacking defined classes and functions. Many Python programs are scripts of this type.

Category 2 (Function-based modular code): Code samples with definitions of standalone functions that act as independent units of code. Such organization of code offers limited encapsulation with potential reliance on global variables and no inherent hiding or protection of the data within functions.

Category 3 (Class-based modular code): Code samples containing defined classes and methods. Most of the larger code samples collected belonged to this category.

2.2 Data Collection

To conduct a comprehensive evaluation of unit test generation tools, we began by gathering a dataset of Python code samples, encompassing a diverse range of projects. Pynguin has a limitation that it does not work well with Python programs that make use of native code, such as Numpy [21][22]. For our comparative analysis, we had to make use of Python projects that did not have a dependency on such libraries.

Initially, we selected about 60 Python projects from open source Github repositories¹, ensuring a diverse and balanced representation while being mindful of Pynguin's limitation. Later we added an additional set of 49 Python Projects from the benchmark data used by Lukasczyk et al. [21]. In total, we have 109 Python projects.

Each project contained multiple files, having import dependencies on one another. For the purpose of our test-generation experiments, we decided to select one core module (i.e. one file) from each project, that we would pass as the prompt input to ChatGPT. We initially limited our selection of core modules to a size of 0-100 lines of code (LOC), which we will call 'Small Code Samples'. We then expanded the scope to consider core module files of sizes ranging from 100 to 300 LOC, we will refer to them as 'Large Code Samples'.

For selecting one core module from each project, we narrowed down our selection to files lying in the required LOC range. From these files, we selected the ones which had the highest McCabe complexity [24] (also known as cyclomatic complexity). For files with similar complexity, we further looked at the number of functions, the richness of logic, and how frequently they were being referenced or imported by other files. Files having high complexity, more function count, and a higher number of import dependencies were chosen as core modules. The goal was to select modules that can be understood on their own without providing prior context, but will also not be too trivial for testing.

We were able to collect a total of 60 core modules under 'Small Code samples', with 20 modules belonging to each of the three categories. And under the 'Large Code Samples', we collected 49 core modules, with 20 modules in Category 2, 23 modules in Category 3 and 6 in Category 1 (very few fell in this category, as most of the projects follow a modular programming approach for easier maintenance).

2.3 Unit Test Generation Tools

We explored the potential of recent tools, Ticoder and Codamosa; however, due to their unavailability for direct experimentation, we opted for Pynguin, a proficient Python unit test generation tool, to further carry out a comprehensive comparison with ChatGPT. We

¹The selected projects from all the GitHub repositories can be found here: <https://github.com/Rey-2001/LLM-nirvana>

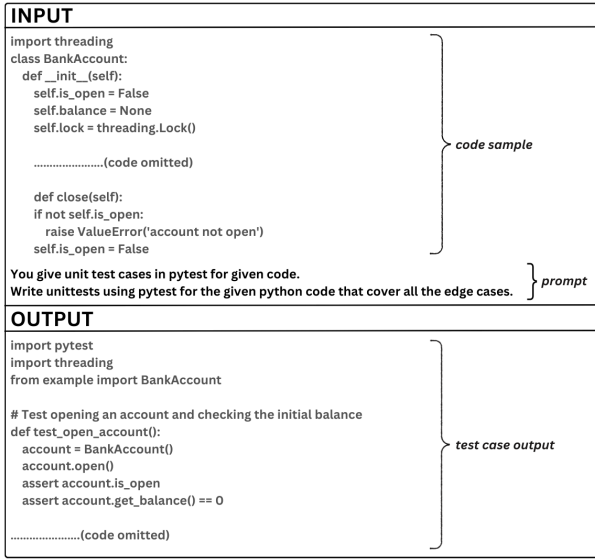


Figure 1: Basic Prompt

engaged ChatGPT and Pynguin to generate unit test cases for each of the identified core modules.

2.4 Prompt Design for ChatGPT

We design our prompt by using clear and descriptive words to capture the intent behind our query, based on the widely acknowledged experience of utilizing ChatGPT[1][2].

Our prompt comprises of two components: i) the Python program, and ii) the descriptive text in natural language outlining the task we aim to accomplish as shown in **Figure 1**. In part i) we provide the whole code of the selected core module (100-300 LOC). We are not providing separate units into ChatGPT, as many studies have[15][30][33][35], as we aim to evaluate ChatGPT's ability to identify units when provided with a complete Python Program. In part ii), we query ChatGPT as follows: "Write Unit tests using Pytest for given Python code that covers all the edge cases."

The ChatGPT-generated test cases are then evaluated for their statement and branch coverage against that of Pynguin-generated test cases; through this, we also find the missed statements by ChatGPT and Pynguin, that the generated test cases are unable to cover, and see if they overlap. Next, we designed a new prompt for ChatGPT that would take in the indices of these missed statements, and ask it to again generate unit tests so as to improve the coverage. Following this, we are also piqued by the possibility of iteratively prompting ChatGPT to keep improving the coverage. We then repeatedly prompt ChatGPT while updating the prompt with the indices of the new set of missed statements after every iteration, till we observe no further improvement in coverage as illustrated in **Figure 2**.

2.5 Evaluation Metrics

The efficacy of the generated unit tests was assessed through a multifaceted approach, employing the following metrics:

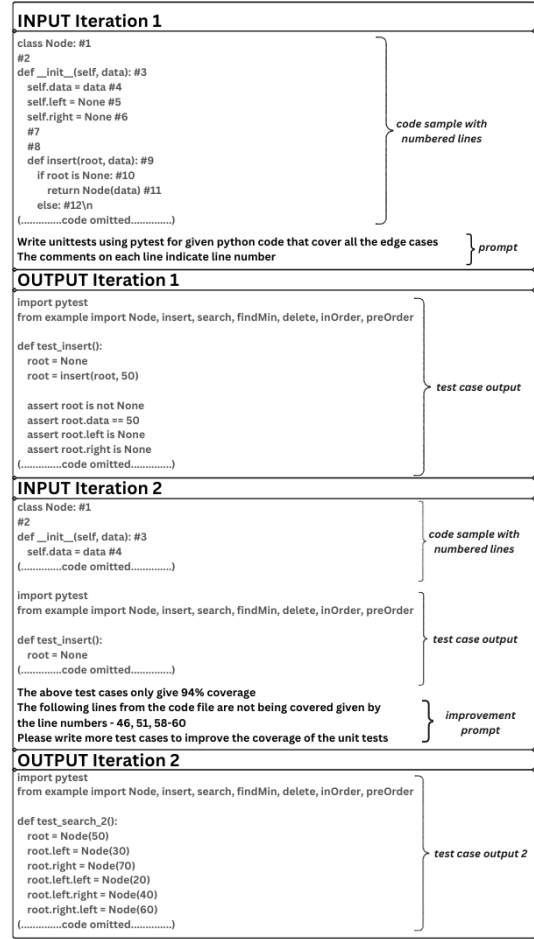


Figure 2: Improvement Prompt

- **Statement Coverage:** Quantifying the extent to which the generated tests covered individual code statements.
- **Branch Coverage:** Evaluating the coverage of various code branches, and gauging the effectiveness of the test suite in exploring different execution paths.
- **Correctness:** Checking if the generated assertions are useful for evaluating the intended functionality of the code, in addition to being correct.

2.6 Experimental Procedure

After gathering the test cases generated by ChatGPT and Pynguin, we compare their performance based on statement and branch coverage. We then try to iteratively prompt ChatGPT to improve its coverage. After reaching the saturation point, where-after no improvement is observed in ChatGPT-generated tests, we evaluate the quality of the ChatGPT-generated test cases by looking at their correctness. We further find out whether any overlap exists in the missed statements between ChatGPT and Pynguin. **Figure 3** shows the workflow of our empirical analysis.

Moving through, we aim to address the research questions in the following order:

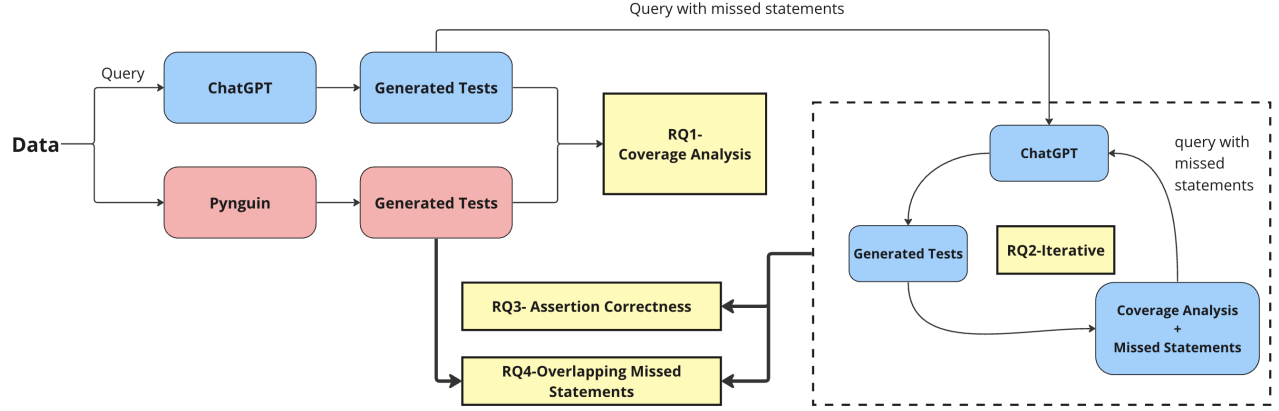


Figure 3: Workflow of our Empirical Analysis

- **RQ1 (Comparative Performance):** We measure the differences in statement coverage and branch coverage between unit tests generated by ChatGPT and Pynguin across different code structures and complexities. We then investigate the correlation, if any, between cyclomatic complexity, code structure, and ChatGPT’s achieved coverage.
- **RQ2 (Performance Saturation and Iterative Improvement):** We then iteratively prompt ChatGPT to improve the coverage for a sample, given the indices of missed statements.
- **RQ3 (Quality Assessment):** We check the generated unit test cases for compilation errors. And among the test cases that are compiling, we check whether their assertions correctly test the intended functionality of the code.
- **RQ4 (Combining Pynguin and ChatGPT for Improved Performance):** In cases where statements are missed, we look at the extent of overlap in missed statements between unit tests generated by ChatGPT and Pynguin. We use this to conclude whether a combination of techniques used by both tools could be a prospective solution for improved performance.

3 Results

3.1 Small Code Samples (0-100 LOC)

For Category 1 (procedural scripts), Pynguin failed to generate any unit tests cases. This can be attributed to the structure of Category 1 samples, where there is a lack of structure and Pynguin is unable to identify distinct units for testing as it relies on properties of modular code. It was noted that ChatGPT provides recommendations for refactoring the category 1 programs into modular units. It first generated the refactored code for the provided code sample and then proceeded to generate the test cases according to the modified code.

For Category 2, there is no significant difference in the statement and branch coverage achieved by Pynguin and ChatGPT. This is also evident from the p-values (threshold = 0.05) obtained after

0-100 LOC	Avg Statement Coverage		Avg Branch Coverage	
	ChatGPT	Pynguin	ChatGPT	Pynguin
Category 1 (original)	0	0	0	0
Category 1 (refactored)	97.45	0	96.85	0
Category 2	93.26	90.3	91.68	90.1
Category 3	91.55	97	89.5	96.15

Table 1: Average statement and branch coverage obtained by ChatGPT & Pynguin for small code samples. Both the tools give comparable performance for Category 2 and Category 3.

performing Independent t-test [17]. p-value for statement coverage is 0.631 while it is 0.807 for branch coverage. Both p-values are higher than the threshold.

For Category 3 also, there is no significant difference in statement and branch coverage achieved by the two tools, given the respective p-values are greater than the threshold: 0.218 and 0.193.

In conclusion, ChatGPT and Pynguin give similar coverage for 0-100 LOC code samples as shown in **Table 1**.

3.2 Large Code Samples (100-300 LOC)

3.2.1 Category-wise Coverage Analysis Since for Category 1, unit test generation is not feasible due to lack of well defined units, and code-refactoring is a wide domain, we limit our coverage analysis to category 2 and 3 for large code samples having 100-300 LOC. For Category 2, we observed that there was no significant difference between the statement and branch coverage achieved by ChatGPT and Pynguin, with respective p-values greater than 0.05 (threshold); 0.169 and 0.195. For Category 3 as well, ChatGPT and Pynguin gave similar coverage, signified by p-values 0.677 and 0.580 for statement and branch coverage respectively. These observations are presented in detail in **Table 2** and **Figure 4**.

To answer **RQ1**: ChatGPT and Pynguin give comparable statement and branch coverage for all 3 categories. Additionally, the McCabe complexity is a metric to evaluate the complexity of a unit

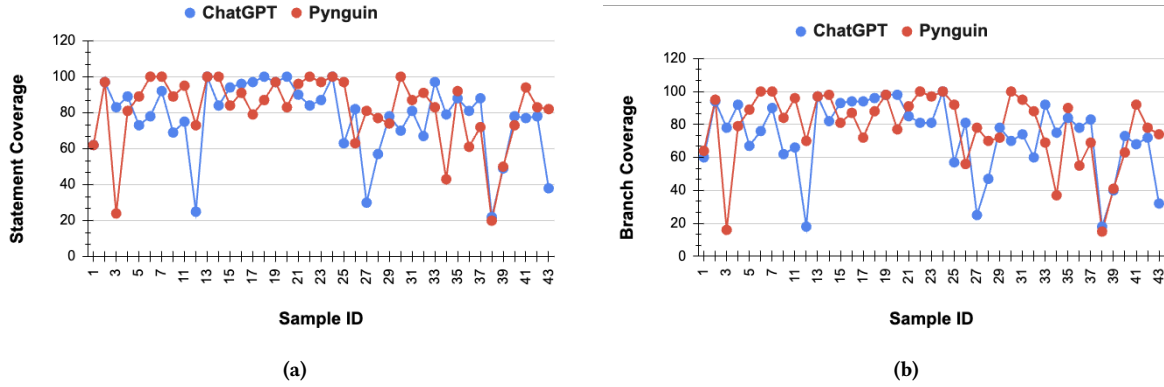


Figure 4: Statement coverage (left) and Branch coverage (right) obtained by ChatGPT (blue) and Pynguin (red) for all code samples (100-300 LOC).

100-300 LOC	Avg Statement Coverage		Avg Branch Coverage	
	ChatGPT	Pynguin	ChatGPT	Pynguin
Category 2	77.44	88.77	74.77	86.22
Category 3	77.4	79.6	73	76.15
Average on all 40 samples	77.43	81.63	73.39	78.36

Table 2: Average statement and branch coverages obtained by ChatGPT & Pynguin for large code samples. Here also, we find ChatGPT's coverage is comparable with Pynguin.

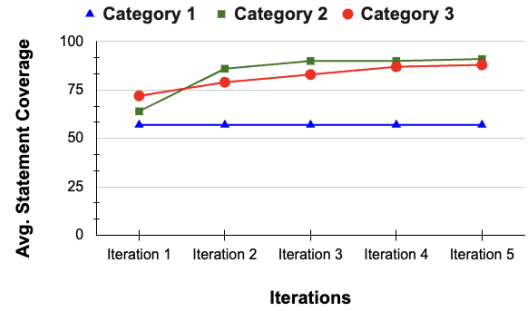


Figure 6: Average statement coverage obtained after each iteration for all of the 3 categories. We see that improvement in coverage saturates at 4 iterations.

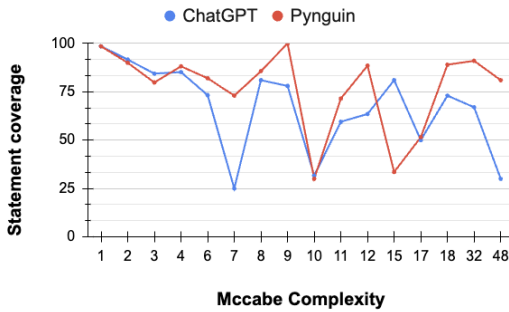


Figure 5: Statement coverage obtained by ChatGPT (blue) and Pynguin (red) for all code samples at different McCabe Complexities.

of code. For a code sample, we assign it the max McCabe complexity, which is the maximum of all the units present in the code. However, plotting the average statement coverage against this complexity measure does not seem to highlight any trend or correlation between coverage achieved by each tool and the maximum McCabe complexity of a code sample as seen in Figure 5.

3.2.2 Iterative improvement in Coverage Till now, the coverage analysis was done on the results obtained from the first iteration of prompting ChatGPT. Providing the missed statements from the

first iteration, as part of the prompt to ChatGPT, we ask it to further improve the coverage for a given code sample. We continue this process till there was no improvement in coverage between consecutive iterations. We had to iteratively prompt ChatGPT for 5 times at most since the coverage for most of the samples converged at iteration 4 as seen in Figure 6.

To answer RQ2: It was observed that the statement coverage in Category 2 and Category 3 increased by 27.95 and 15.25 respectively on average. For Category 1, we found that despite the number of iterations there was no improvement in coverage at any step as seen in Table 3.

3.2.3 Correctness To assess the correctness of assertions produced by ChatGPT, we examine the various error categories in the following manner: i) whether the test cases are compiling, ii) the percentage of passing assertions among the compiling test cases, and iii) the nature of errors encountered for assertions that fail. It is crucial to emphasize that the source code snippets used for generating these assertions are derived from well-established Python projects publicly available for general use, meaning that these projects must have been thoroughly tested to perform what they were intended to do. This implies that the generated test cases by ChatGPT should

Avg Coverage	Iter 1	Iter 5	Best Coverage	Diff in best & least cov	Med iters for Cov plateau
Category 1 (100-300 LOC)	56.833	56.833	56.833	0	1
Category 2 (100-300 LOC)	63.6	90.5	91.55	27.95	4
Category 3 (100-300 LOC)	72.45	87.7	87.7	15.25	4

Table 3: Iterative Improvement. This table shows the Average statement coverages achieved in iteration 1 and 5 for all 3 categories. Also depicts the average improvement in statement coverage after 5 iterations and the median number of iterations it takes to reach the saturation point in coverage improvement. Best coverage: Average of Best Overall Coverage over all iterations. Diff in best & least cov: Average Difference in best & least coverage. Med iters for Cov plateau: Median iterations to achieve Coverage saturation.

ChatGPT Generated % of incorrect assertions	Category 1	Category 2	Category 3
All Incorrect	57.75	39.4	27.67
Assertion Error	86.93	78.30	70.96
Try/Except Error	0	2.68	3.66
Runtime error	13.07	19.02	25.38

Table 4: Percentage of Incorrect Assertions. This table gives the average percentage of incorrect assertions generated by ChatGPT for all 3 categories of code samples. Also specifies the distribution across 3 causes of failing assertions: i) Assertion-Error: occurs when the asserted condition is not met ii) Error in try/except block: occurs when an exception was expected to be raised, but it wasn't raised iii) Runtime Error : occurs while the program is running after being successfully compiled.

effectively capture the intended functionality of the code, and the corresponding assertions should pass if they correctly test the logic of the code. However, if any assertions do fail, it indicates that those assertions fail to test the intended functionality of the code and thus are not correct.

To answer **RQ3**: As depicted in **Table 4**, about 39% of generated assertions are incorrect on average for Category 2 while 28% of assertions are incorrect on average for Category 3. Separately, we also checked the correctness of assertions for Category 1 samples, for which ChatGPT had provided refactored code, and found that 58% of the generated assertions were incorrect. The decrease in percentage of incorrect assertions as we go from category 1 to category 3, may imply that ChatGPT's ability to generate correct assertions is higher for programs with well defined structure, possibly due to presence of more coherent and meaningful units in the code.

3.2.4 Overlapping Missed Statements We looked at the number of overlapping statements that were commonly being missed by ChatGPT and Pynguin. We observed, on average, out of a combined total of 50 missed statements, around 17 were common to both, which means the overlap is significantly lower than the total number of missed statements.

Intersecting missed	17.78
Union missed	50.33
Minimum missed	20.29
Minimum covered	2.51
Maximum covered	31.64

Table 5: Summary Statistics of missed overlapping statements. This table shows summary statistics on statements not covered through the tests generated by ChatGPT and Pynguin. Intersecting missed: average number of common statements that were missed by both ChatGPT and Pynguin. Union missed: average number of statements that were missed by either ChatGPT or Pynguin. Minimum missed: Min of (average number of statements missed by ChatGPT, average number of statements missed by Pynguin). Minimum covered: The number of statements that are atleast covered after combining ChatGPT and Pynguin on average. Maximum covered: It is the maximum possible number of statements that should be covered after combining Chatgpt and Pynguin on average.

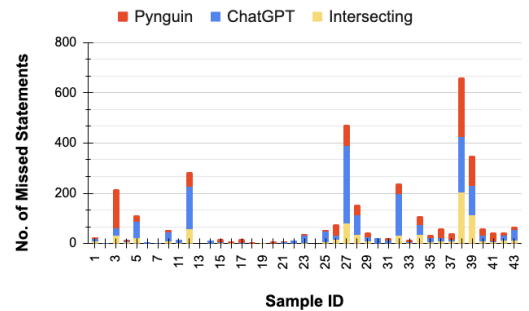


Figure 7: Missed Statements by ChatGPT, Pynguin and their intersection for each code sample.

To answer **RQ4**: If we were to combine ChatGPT and Pynguin, a possible logical inference would be that, the only missed statements would be the ones which were overlapping, and rest all the statements, which were earlier being individually missed by ChatGPT and Pynguin, will now be covered. For example, based on our evaluation, a combination of the two tools will result in 31 more statements being covered. This is illustrated in **Table 5** and **Figure 7**. This is also in line with the findings of CODAMOSA [19] which proposes that a combination of SBST and LLM can lead to better coverage.

4 Related Work

In this section, we briefly discuss some of the existing tools and techniques for generating unit tests.

Search-based software testing (SBST) techniques: These techniques [4, 5, 9, 13, 31] turn testing into an optimization problem to generate unit test cases. The goal of SBST is to generate optimal

test suites that improve code coverage and efficiently reveal program errors by utilizing algorithms to traverse problem space. This approach shows potential in lowering the quantity of test cases needed while preserving reliable detection of errors. EvoSuite [12] is an automated test generation tool which utilizes SBST. It takes a Java class or method as input, uses search-based algorithms to create a test suite meeting specified criteria (e.g., code or branch coverage), and evaluates test fitness. Through iterative processes of variation, selection, and optimization, EvoSuite generates JUnit test cases and provides a report on the effectiveness of the produced test suite based on metrics like code coverage and mutation score.

Pynguin [21] is another tool which utilizes SBST for generating unit tests in Python programming language. The variable types are dynamically assigned at runtime in Python which makes it difficult to generate unit tests. Pynguin examines a Python module to gather details about the declared classes, functions, and methods. It then creates a test cluster with every relevant information about the module being tested, and during the generation process, chooses classes, methods, and functions from the test cluster to build the test cases. We use Pynguin as our baseline for comparing the suitability and effectiveness of LLMs in unit test generation.

Randomized test generation techniques: Randoop [26] uses feedback-directed random testing to generate test cases. The basic idea behind this technique is to generate random sequences of method calls and inputs that exercise different paths through the program. As the test runs, Randoop collects information about the code coverage achieved by the test, as well as any exceptions that are thrown. Based on this feedback, Randoop tries to generate more test cases that are likely to increase code coverage or trigger previously unexplored behaviour.

AI-based techniques: Ticoder [18] presents an innovative Test-Driven User-Intent Formalisation (TDUIF) approach for generating code from natural language with minimal formal semantics. Their system, TICODER, demonstrates improved code generation accuracy and the ability to create non-trivial functional unit tests aligned with user intent through minimal user queries. TOGA [11] introduces a neural method for Test Oracle Generation, using a transformer-based approach to infer exceptional and assertion test oracles based on the context of the focal method.

CODAMOSA [19] introduces an algorithm that enhances Search-Based Software Testing (SBST) by utilizing pre-trained large language models (LLMs) like OpenAI's Codex [25]. The approach combines test case generation with mutation to produce high-coverage test cases and requests Codex to provide sample test cases for under-covered functions. Our paper confirms some of the findings from CODAMOSA. For instance, our results also show that a combination of LLM and Pynguin (SBST-based) can lead to a better coverage. At the same time, CODAMOSA does not explore some of the research questions which we explored in this paper such as (1) How correct are the assertions generated by LLMs? (2) Do the LLM-generated tests align with the intended functionality of the code? (3) How does the performance of LLM improve over multiple iterations of prompting?

5 Conclusion

In this study, we discovered that ChatGPT and Pynguin demonstrated nearly identical coverage for both small and large code samples, with no statistically significant differences in average coverages across all categories. When iteratively prompting ChatGPT to enhance coverage, by providing the indices of missed statements from previous iteration, improvements were notable for categories 2 and 3, reaching saturation at 4 iterations, while no improvement occurred for category 1.

Notably, individually missed statements by both tools showed minimal overlap, hinting at the potential for a combined approach to yield higher coverage. Lastly, our assessment of the correctness of ChatGPT-generated tests revealed a decreasing trend in the percentage of incorrect assertions from Category 1 to 3, which could possibly suggest that assertions generated by ChatGPT are more effective in cases where code units are well defined.

ChatGPT operates with a focus on understanding and generating content in natural language rather than being explicitly tailored for programming languages. While ChatGPT may be capable of achieving high statement coverage in the generated unit tests, a high percentage of the assertions within those tests might be incorrect. A more effective approach to generating correct assertions would be based on the actual semantics of the code. This presents a concern that ChatGPT may prioritize coverage over the accuracy of the generated assertions, which is a potential limitation in using ChatGPT for generating unit tests, and a more semantic-based approach might be needed for generating accurate assertions. Future research endeavors could delve into several promising avenues based on the findings of this study. Firstly, exploring how ChatGPT refactors code from procedural scripts and assessing whether the refactored code preserves the original functionality could provide valuable insights into the model's code transformation capabilities. Additionally, investigating the scalability of ChatGPT and Pynguin to larger codebases and more complex projects may offer a broader understanding of their performance in real-world scenarios. Furthermore, a comprehensive exploration of the combined use of ChatGPT and Pynguin, considering their complementary strengths, could be undertaken to maximize test coverage and effectiveness. Lastly, examining the generalizability of our observations across diverse programming languages and application domains would contribute to a more comprehensive understanding of the applicability and limitations of these tools.

References

- [1] [n. d.]. OpenAI Platform. <https://platform.openai.com>
- [2] 2022. OpenAI's ChatGPT: Optimizing Language Models for Dialogue – cloudHQ. <https://blog.cloudhq.net/openai-chatgpt-optimizing-language-models-for-dialogue/>
- [3] Toufique Ahmed and Premkumar Devanbu. 2023. Few-Shot Training LLMs for Project-Specific Code-Summarization. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering* (Rochester, MI, USA) (ASE '22). Association for Computing Machinery, New York, NY, USA, Article 177, 5 pages. <https://doi.org/10.1145/3551349.3559555>
- [4] James H. Andrews, Tim Menzies, and Felix C.H. Li. 2011. Genetic Algorithms for Randomized Unit Testing. *IEEE Transactions on Software Engineering* 37, 1 (2011), 80–94. <https://doi.org/10.1109/TSE.2010.46>
- [5] Luciano Baresi and Matteo Miraz. 2010. TestFul: Automatic Unit-Test Generation for Java Classes. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - Volume 2* (Cape Town, South Africa) (ICSE '10). Association for Computing Machinery, New York, NY, USA, 281–284. <https://doi.org/10.1145/1810295.1810353>

- [6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. *arXiv:2005.14165* [cs.CL]
- [7] Paul F. Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep Reinforcement Learning from Human Preferences. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/d5e2c0adad503c91f91df240d0cd4e49-Paper.pdf
- [8] Christoph Csallner and Yannis Smaragdakis. 2004. JCrasher: An automatic robustness tester for Java. *Softw., Pract. Exper.* 34 (09 2004), 1025–1050. <https://doi.org/10.1002/spe.602>
- [9] Pouria Derakhshanfar, Xavier Devroey, and Andy Zaidman. 2022. Basic Block Coverage for Search-based Unit Testing and Crash Reproduction. *arXiv:2203.02337* [cs.SE]
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv:1810.04805* [cs.CL]
- [11] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. 2022. TOGA: a neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM. <https://doi.org/10.1145/3510003.3510141>
- [12] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: Automatic Test Suite Generation for Object-Oriented Software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (Szeged, Hungary) (ESEC/FSE '11)*. Association for Computing Machinery, New York, NY, USA, 416–419. <https://doi.org/10.1145/2025113.2025179>
- [13] Gordon Fraser and Andreas Zeller. 2012. Mutation-Driven Generation of Unit Tests and Oracles. *IEEE Transactions on Software Engineering* 38, 2 (2012), 278–292. <https://doi.org/10.1109/TSE.2011.93>
- [14] Sepideh Kashefi Gargari and Mohammad Reza Keyvanpour. 2021. SBST challenges from the perspective of the test techniques. In *2021 12th International Conference on Information and Knowledge Technology (IKT)*. 119–123. <https://doi.org/10.1109/IKT54664.2021.9685297>
- [15] Vitor Guilherme and Auri Vincenzi. 2023. An Initial Investigation of ChatGPT Unit Test Generation Capability. In *Proceedings of the 8th Brazilian Symposium on Systematic and Automated Software Testing (<conf-loc>, <city>Campo Grande, MS</city>, <country>Brazil</country>, </conf-loc>)* (SAST '23). Association for Computing Machinery, New York, NY, USA, 15–24. <https://doi.org/10.1145/3624032.3624035>
- [16] Mark Harman, Yue Jia, and Yuanyuan Zhang. 2015. Achievements, Open Problems and Challenges for Search Based Software Testing. In *2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)*. 1–12. <https://doi.org/10.1109/ICST.2015.7102580>
- [17] Kim Tae Kyun. 2015. T test as a parametric statistic. *kja* 68, 6 (2015), 540–546. <https://doi.org/10.4097/kjae.2015.68.6.540> *arXiv:https://www.e-sciencecentral.org/articles/?scid=1156170*
- [18] Shuvendu K. Lahiri, Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, Madanlal Musuvathi, Piali Choudhury, Curtis von Veh, Jeevana Priya Inala, Chenglong Wang, and Jianfeng Gao. 2023. Interactive Code Generation via Test-Driven User-Intent Formalization. *arXiv:2208.05950* [cs.SE]
- [19] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. 919–931. <https://doi.org/10.1109/ICSE48619.2023.00085>
- [20] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2019. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. *arXiv:1910.13461* [cs.CL]
- [21] Stephan Lukaszczuk and Gordon Fraser. 2022. Penguin: automated unit test generation for Python. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings (ICSE '22)*. ACM. <https://doi.org/10.1145/3510454.3516829>
- [22] Stephan Lukaszczuk, Florian Kroiß, and Gordon Fraser. 2023. An empirical study of automated unit test generation for Python. *Empirical Software Engineering* 28, 2 (Jan. 2023), 36. <https://doi.org/10.1007/s10664-022-10248-w>
- [23] Lei Ma, Cyrille Artho, Cheng Zhang, Hiroyuki Sato, Johannes Gmeiner, and Rudolf Ramler. 2015. GRT: Program-Analysis-Guided Random Testing (T). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 212–223. <https://doi.org/10.1109/ASE.2015.49>
- [24] T.J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320. <https://doi.org/10.1109/TSE.1976.233837>
- [25] OpenAI. 2023. OpenAI Codex. <https://openai.com/blog/openai-codex>
- [26] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: Feedback-Directed Random Testing for Java. In *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion (Montreal, Quebec, Canada) (OOPSLA '07)*. Association for Computing Machinery, New York, NY, USA, 815–816. <https://doi.org/10.1145/1297846.1297902>
- [27] Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball. 2007. Feedback-Directed Random Test Generation. In *29th International Conference on Software Engineering (ICSE '07)*. 75–84. <https://doi.org/10.1109/ICSE.2007.37>
- [28] Abdelilah Sakti, Gilles Pesant, and Yann-Gaël Guéhéneuc. 2015. Instance Generator and Problem Representation to Improve Object Oriented Code Coverage. *IEEE Transactions on Software Engineering* 41, 3 (2015), 294–313. <https://doi.org/10.1109/TSE.2014.2363479>
- [29] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *arXiv:2302.06527* [cs.SE]
- [30] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. 2023. ChatGPT vs SBST: A Comparative Assessment of Unit Test Suite Generation. *arXiv:2307.00588* [cs.SE]
- [31] Paolo Tonella. 2004. Evolutionary Testing of Classes. *SIGSOFT Softw. Eng. Notes* 29, 4 (jul 2004), 119–128. <https://doi.org/10.1145/1013886.1007528>
- [32] Kingyao Wang, Sha Li, and Heng Ji. 2023. Code4Struct: Code Generation for Few-Shot Event Structure Prediction. *arXiv:2210.12810* [cs.CL]
- [33] Zhuokui Xie, Yinghao Chen, Chen Zhi, Shuiguang Deng, and Jianwei Yin. 2023. ChatUnitTest: a ChatGPT-based automated unit test generation tool. *arXiv:2305.04764* [cs.SE]
- [34] Shengcheng Yu, Chunrong Fang, Yuchen Ling, Chentian Wu, and Zhenyu Chen. 2023. LLM for Test Script Generation and Migration: Challenges, Capabilities, and Opportunities. *arXiv:2309.13574* [cs.SE]
- [35] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation. *arXiv:2305.04207* [cs.SE]